

Pythonic Data Structures

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Exercise: Let's write these classes and corresponding tests

A class that represents a message in a group chat

- Constructor takes the contents of the message and the writer's user ID, and stores both in attributes
- Constructor also stores current time
 - `from datetime import datetime, UTC` and `now_utc = datetime.now(UTC)`
- A `display()` method that returns a `str` with the userID, timestamp, and message

A class that represents a group chat

- Constructor takes no arguments, but stores a timestamp and starts a log of messages
- An `add_message()` method that takes a message and user ID and adds a message to the log
- A `display()` method that returns a `str` with all messages, formatted nicely

Two ways to create lists

Create lists by listing their elements

```
my_nums: list[int] = [6, 7, 8, 9]
words: list[str] = [
    'never',
    'gonna',
    'give',
    'you',
    'up'
]
```

`split()`: split a `str` into separate words

```
words: list[str] = 'never gonna give you up'.split()
```

```
['never', 'gonna', 'give', 'you', 'up']
```

Optional parameter `sep` to use something other than whitespaces:

```
lyric = 'never gonna give you up'
words: list[str] = lyric.split(sep = 'e')
```

```
['n', 'v', 'r gonna giv', ' you up']
```

Exercise: Let's write a function that does the opposite of `split()`

It takes a `list[str]` and `delimiter: str`, and combines it into a `str`

List indices

In Python (and most other programming languages), lists are indexed starting with 0 on the very left, and increasing as it goes to the right.

```
words: list[str] = 'never gonna give you up'.split()
second_word: str = words[1]
first_word: str = words[0]

first_three_words: list[str] = [first_word, second_word, words[2]]
print(first_three_words)    # ['never', 'gonna', 'give']
```

Remember: this means that the last index in the list is its length minus one

What happens if we try to access an index that is larger than that:

```
IndexError: list index out of range
```

Built-in function that does that for us

join(): combine a list of **str** into a single **str**

```
phrase: str = ' '.join(
    ['never', 'gonna', 'give',
     'you', 'up'])

also_phrase: str = 'e'.join(
    ['n', 'v', 'r gonna giv',
     ' you up'])
```

str to the left of the **.join()** is used as "glue" or "fenceposts" between the combined **str**s

Python also has a second set of indices starting with -1 on the very right, and counting down (more negative) as it steps leftward.

```
last_word: str = words[-1]
penultimate_word: str = words[-2]
print(f'{penultimate_word} {last_word}')
```

you up

Poll: What does this evaluate to?

```
'never gonna give you up'.split()[-4]
```

1. never
2. gonna
3. give
4. you
5. up

Exercise: Let's write a function that takes a `list` and returns the last `n` elements as another `list`

Use negative indices

Use generic types

If the list is too short, raise a `ValueError`

Indexing works the same on strings (::)

Exercise: Let's write a function that takes a `str` and returns a list of all the character n-grams (sliding window of `n` characters)

List slices

Use "slicing" to get a sub-list (a contiguous part of the list)

```
letters: list[str] = list('abcdefghijklmnopqrstuvwxyz')
print(letters)  # ['a', 'b', 'c', 'd', ..., 'x', 'y', 'z']

second_third_fourth_letters: list[str] = letters[2:5]
print(second_third_fourth_letters)  # ['c', 'd', 'e']
```

In brackets: starting index (inclusive) and stopping index (exclusive)

- The start must be to the left of the stop.
- Both must be valid indices.
- It returns a new list that is a copy of that part of the original list, without modifying the original list.

Omit starting index: start at the very beginning of the list

```
letters: list[str] = list('abcdefghijklmnopqrstuvwxyz')  
print(letters[:4]) # ['a', 'b', 'c', 'd']
```

Omit stopping index: end at the very end of the list

```
letters: list[str] = list('abcdefghijklmnopqrstuvwxyz')  
print(letters[20:]) # ['u', 'v', 'w', 'x', 'y', 'z']
```

Omit both: create a copy of the entire list

```
letters: list[str] = list('abcdefghijklmnopqrstuvwxyz')  
print(''.join(letters[:])) # abcdefghijklmnopqrstuvwxyz
```

Poll: What is printed?

```
letters: list[str] = list('abcdefghijklmnopqrstuvwxyz')  
print(f'{letters[-len(letters)]} {''.join(letters[23:])} {letters[-1]}')
```

1. a wxyz z

2. a xyz z

3. z wxyz z

4. z xyz z

Modifying a list

Replace elements:

```
my_nums[-1] = 900
```

Insert or append elements (which makes the list longer):

```
my_nums: list[int] = [5, 6, 7, 8, 9]

my_nums.insert(3, 600)
print(my_nums)  # [5, 6, 7, 600, 8, 9]

my_nums.append(10)
print(my_nums)  # [5, 6, 7, 600, 8, 9, 10]
```

Modifying a list

Replace elements:

```
my_nums[-1] = 900
```

Insert or append elements (which makes the list longer):

```
my_nums: list[int] = [5, 6, 7, 8, 9]
my_nums.insert(3, 600)
print(my_nums) # [5, 6, 7, 600, 8, 9]
my_nums.append(10)
print(my_nums) # [5, 6, 7, 600, 8, 9, 10]
```

Append multiple elements at once using `extend()`:

```
my_nums.extend([700, 800, 900])
print(my_nums) # [5, 6, 7, 600, 8, 9, 10, 700, 800, 900]
```

Poll: Which function takes a list of integers, and returns a copy of it, but without the negative numbers?

a)

```
def positive_copy(nums: list[int]) -> list[int]:  
    result: list[int] = list()  
    for i in nums:  
        if i >= 0:  
            result.append(i)  
    return result
```

c)

```
def positive_copy(nums: list[int]) -> list[int]:  
    result: list[int] = list()  
    for i in range(len(nums)):  
        if nums[i] >= 0:  
            result.append(nums[i])  
    return result
```

b)

```
def positive_copy(nums: list[int]) -> list[int]:  
    result: list[int] = list()  
    for i in range(len(nums)):  
        if i >= 0:  
            result.append(i)  
    return result
```

d)

```
def positive_copy(nums: list[int]) -> list[int]:  
    result: list[int] = list()  
    for i in range(0, -len(nums), -1):  
        if nums[i - 1] >= 0:  
            result.insert(0, nums[i - 1])  
    return result
```

List comprehension

- Creating an empty list and adding elements one by one is not efficient.
- It's also hard to read.
- Instead, we can use **list comprehension** to let Python optimize it.

Use list comprehension to make a copy of the list, but with each element increased by one:

```
my_nums: list[int] = [6, 7, 8, 9]

increased_nums: list[int] = [i + 1 for i in my_nums] # list comprehension

print(increased_nums) # [7, 8, 9, 10]
```

One way to look at this format: that we moved the body of a `for` loop to right before the `for` (after the opening bracket `[`).

If we want the resulting list to **filter elements**, we add the `if` clause after the `for` clause.

`positive_copy()` using list comprehension:

```
def positive_copy(nums: list[int]) -> list[int]:  
    return [i for i in nums if i >= 0]
```

Poll: Which of these is a one-line version of the inside of our favorite `sarcasm()` function?

1. `return ''.join([character.upper() for character in phrase if random() < 0.5])`
2. `return ''.join([character.upper() if random() < 0.5 for character in phrase])`
3. `return ''.join([character.upper() if random() else character.lower() for character in phrase])`
4. `return ''.join([character.upper() if random() < 0.5 else character.lower() for character in phrase])`

List comprehension can be used for things that aren't lists

Iterate over a string and create a set:

```
phrase: str = 'never gonna give you up'

letters: set[str] = {letter.lower() for letter in phrase}

print(letters)  # {'v', 'g', 'i', 'o', 'n', 'a', 'y', 'p', 'u', 'e', 'r', ' '}
```

It is always up to you to decide which version is easiest to read for your code. Sometimes, a basic `for` loop is more readable.

Poll: What does this function do (other than confuse)?

```
def something(docs: list[str]) -> int:
    """Confuses students.

    Parameters:
        docs : list[str]
            A list of confusing strings, each more ridiculous than the last

    Returns:
        int
            A confusing number
    """
    return sum(len(open(doc, encoding="utf-8").read()) for doc in docs)
```

Tuple: like a list, but immutable (i.e., its contents cannot be changed after declaration)

What is the `type` of the collection of arguments `*args`? A `tuple`!

```
def print_args(*args: T) -> None:
    """Print each argument on a separate line"""
    for item in args:
        print(item)

print_args(1, 2, 3)
```

```
nums: tuple[int, int, int] = (1, 2, 3)

my_long_tuple: tuple[int, ...] = tuple([-i for i in range(50)])
```

Tuple: like a list, but immutable (i.e., its contents cannot be changed after declaration)

We can sort a list in-place, but not a tuple.

```
my_list.sort()
print(my_list) # [-400, 1, 2, 3]

my_tuple.sort() # impossible
```

Poll: Tuples and lists are very similar, but we can't modify tuples. Which of these collections should be a tuple instead of a list?

1. Students registered for a course
2. Cats in a shelter
3. The seven days of the week
4. Driving directions from school to the airport (turn left, drive 2 miles, ...)

The tricky part:

- The tuple is immutable
- The variable `my_tuple` (the pointer to the location in the computer's memory) is a mutable variable

We can't mutate the tuple, but we can re-assign the variable `my_tuple` to a sorted version of the same tuple:

```
my_long_tuple: tuple[int, ...] = tuple([-i for i in range(5)])
my_long_tuple = tuple(sorted(my_long_tuple))
print(my_long_tuple)           # (-4, -3, -2, -1, 0)
```

Poll: Which ONE is not allowed? (Hint: `str`s are immutable)

```
my_str: str = 'mini'
```

1. `print(my_str.upper())`
2. `my_str = my_str.upper()`
3. `my_str = 'MINI'`
4. `my_str[0] = 'B'`

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**